Nama: Nafisah Sekar Ayu

NIM: H1D024087

Shift KRS: F

Shift Akhir: C

TUGAS PERTEMUAN 4

1. Kode

```
import tkinter as tk
from tkinter import messagebox

# ========================
# DATA GEJALA
# ========================
gejala_list = [
    ("G01", "Apakah komputer sering terasa lambat saat digunakan?"),
    ("G02", "Apakah aplikasi sering error atau crash?"),
    ("G03", "Apakah komputer tiba-tiba mati sendiri?"),
    ("G04", "Apakah terdengar bunyi beep saat dinyalakan?"),
    ("G05", "Apakah perangkat terasa sangat panas?"),
    ("G06", "Apakah layar tidak menampilkan gambar?"),
    ("G07", "Apakah kipas berbunyi sangat keras?"),
    ("G08", "Apakah harddisk tidak terbaca?"),
    ("G09", "Apakah proses booting sangat lama?"),
    ("G10", "Apakah file sering rusak atau tidak bisa dibuka?")
]

# ========================
# KNOWLEDGE BASE
# ========================
rules = {
    "Gangguan RAM": {
        "kode": ["G01", "G02", "G04"],
        "solusi": "Bersihkan pin RAM menggunakan penghapus, lalu pasang
kembali atau ganti RAM."
    },
    "Kerusakan Power Supply": {
        "kode": ["G03", "G06"],
        "solusi": "Periksa kondisi PSU dan pastikan kabel daya terhubung
dengan baik."
    },
    "Overheat Prosesor": {
        "kode": ["G05", "G07", "G03"],
        "solusi": "Bersihkan kipas dan ganti thermal paste pada prosesor."
    },
    "Kerusakan VGA": {
        "kode": ["G06", "G02"],
```

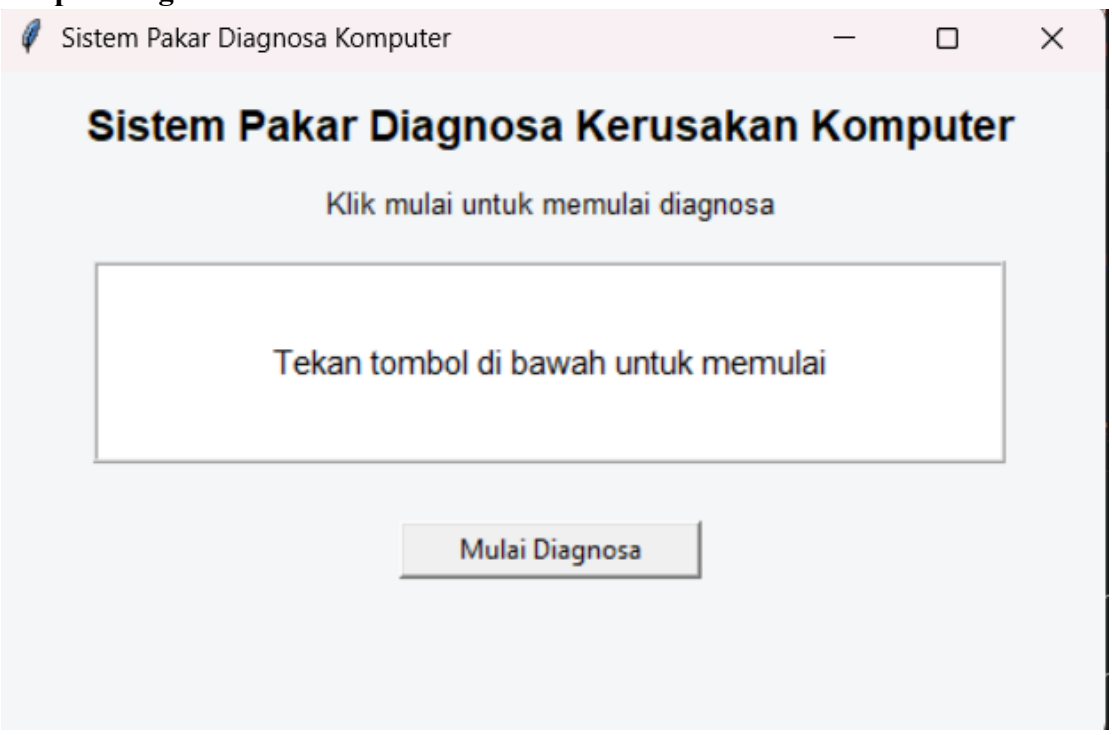
```
        "solusi": "Periksa dan pasang ulang VGA atau ganti jika
diperlukan."
    },
    "Kerusakan Harddisk": {
        "kode": ["G08", "G09", "G10"],
        "solusi": "Segera backup data dan lakukan pengecekan atau
penggantian harddisk."
    }
}

# ========================
# STATE
# ========================
index = 0
jawaban_user = set()

# ========================
# FUNGSI
# ========================
def mulai():
    global index, jawaban_user
    index = 0
    jawaban_user.clear()

    btn_mulai.pack_forget()
    btn_ya.pack(side="left", padx=10)
    btn_tidak.pack(side="right", padx=10)

    tampil_pertanyaan()

def tampil_pertanyaan():
    if index < len(gejala_list):
        kode, teks = gejala_list[index]
        label_info.config(text=f"Pertanyaan {index+1} dari
{len(gejala_list)}")
        label_pertanyaan.config(text=teks)
    else:
        tampilkan_hasil()

def jawab(pilihan):
    global index
    kode, _ = gejala_list[index]

    if pilihan:
        jawaban_user.add(kode)

    index += 1
    tampil_pertanyaan()
```

```python
def tampilkan_hasil():
    # Hitung kecocokan untuk setiap kerusakan
    skor_kerusakan = []  # list of (nama, cocok, total, solusi)
    for nama, data in rules.items():
        cocok = sum(1 for k in data["kode"] if k in jawaban_user)
        total = len(data["kode"])
        skor_kerusakan.append((nama, cocok, total, data["solusi"]))

    # Pisahkan yang cocok sempurna (cocok == total)
    sempurna = [(nama, cocok, total, solusi) for (nama, cocok, total,
solusi) in skor_kerusakan if cocok == total]

    if sempurna:
        # Tampilkan semua kerusakan yang cocok 100%
        hasil = "=== HASIL DIAGNOSA ===\n\n"
        for nama, _, _, solusi in sempurna:
            hasil += f"Kerusakan: {nama}\nSolusi: {solusi}\n\n"
        messagebox.showinfo("Hasil Diagnosa", hasil.strip())
    else:
        # Cari skor tertinggi (max cocok)
        if not skor_kerusakan:
            messagebox.showinfo("Hasil Diagnosa", "Tidak dapat mendiagnosa
kerusakan.")
            reset()
            return

        max_cocok = max(skor_kerusakan, key=lambda x: x[1])[1]
        if max_cocok == 0:
            messagebox.showinfo("Hasil Diagnosa", "Tidak ada gejala yang
cocok dengan basis pengetahuan.")
            reset()
            return

        # Ambil semua kerusakan dengan skor tertinggi
        terbaik = [(nama, cocok, total, solusi) for (nama, cocok, total,
solusi) in skor_kerusakan if cocok == max_cocok]

        hasil = "=== HASIL DIAGNOSA (Tidak ada kecocokan sempurna) ===\n\n"
        for nama, cocok, total, solusi in terbaik:
            persen = (cocok / total) * 100
            hasil += f"Kerusakan: {nama}\nTingkat kecocokan:
{persen:.0f}%\nSolusi: {solusi}\n\n"
        messagebox.showinfo("Hasil Diagnosa", hasil.strip())

    reset()

def reset():
```

```
    btn_ya.pack_forget()
    btn_tidak.pack_forget()
    btn_mulai.pack(pady=10)

    label_info.config(text="Klik mulai untuk memulai diagnosa")
    label_pertanyaan.config(text="Tekan tombol di bawah untuk memulai")

# ========================
# GUI
# ========================
root = tk.Tk()
root.title("Sistem Pakar Diagnosa Komputer")
root.geometry("500x300")
root.config(bg="#f4f6f7")

judul = tk.Label(root,
                 text="Sistem Pakar Diagnosa Kerusakan Komputer",
                 font=("Arial", 14, "bold"),
                 bg="#f4f6f7")
judul.pack(pady=10)

label_info = tk.Label(root,
                      text="Klik mulai untuk memulai diagnosa",
                      font=("Arial", 10),
                      bg="#f4f6f7")
label_info.pack()

label_pertanyaan = tk.Label(root,
                            text="Tekan tombol di bawah untuk memulai",
                            font=("Arial", 11),
                            bg="white",
                            width=45,
                            height=5,
                            wraplength=380,
                            relief="ridge",
                            justify="center")
label_pertanyaan.pack(pady=15)

frame = tk.Frame(root, bg="#f4f6f7")
frame.pack()

btn_ya = tk.Button(frame, text="Ya", width=12,
                   command=lambda: jawab(True))

btn_tidak = tk.Button(frame, text="Tidak", width=12,
                      command=lambda: jawab(False))

btn_mulai = tk.Button(root,
```

```
                        text="Mulai Diagnosa",
                        width=18,
                        command=mulai)
btn_mulai.pack(pady=10)

root.mainloop()
```

2. **Output Program**



3. **Penjelasan Kode**

Program ini dibangun menggunakan bahasa Python dengan antarmuka grafis (GUI) berbasis Tkinter. Tujuan program adalah mendiagnosis kerusakan pada komputer/laptop berdasarkan gejala-gejala yang dipilih pengguna melalui serangkaian pertanyaan ya/tidak.

Struktur Knowledge Base

Basis pengetahuan dibagi menjadi dua komponen utama: daftar gejala dan daftar aturan kerusakan.

Daftar Gejala (gejala_list)

python

```
gejala_list = [
    ("G01", "Apakah komputer sering terasa lambat saat digunakan?"),
    ("G02", "Apakah aplikasi sering error atau crash?"),
    ...
]
```

Setiap elemen adalah tuple berisi kode gejala (string unik) dan kalimat pertanyaan yang akan ditampilkan ke pengguna. Pemilihan kode (G01, G02, ...) memudahkan pencocokan dengan aturan.

Daftar Aturan Kerusakan (rules)

python
```
rules = {
    "Gangguan RAM": {
        "kode": ["G01", "G02", "G04"],
        "solusi": "Bersihkan pin RAM menggunakan penghapus, lalu pasang kembali atau ganti RAM."
    },
    ...
}
```
Struktur ini adalah dictionary bersarang. Key utama adalah nama kerusakan (5 jenis: Gangguan RAM, Kerusakan Power Supply, Overheat Prosesor, Kerusakan VGA, Kerusakan Harddisk). Nilai dari setiap key adalah dictionary dengan dua field:

- "kode" : list kode gejala yang mengindikasikan kerusakan tersebut. Hubungan antar gejala dianggap sebagai **logika AND** implisit (semakin banyak gejala cocok, semakin tinggi kecocokan).
- "solusi" : teks rekomendasi tindakan perbaikan.

Dengan struktur ini, basis pengetahuan bersifat deklaratif dan mudah diperluas tanpa mengubah kode logika inferensi

Mesin Inferensi

Mesin inferensi tidak menggunakan rantai if-else yang panjang, melainkan melakukan pencocokan berbasis himpunan. Saat pengguna selesai menjawab semua pertanyaan, program menyimpan kode gejala yang dijawab "Ya" ke dalam set `jawaban_user`. Untuk setiap kerusakan dalam `rules`, program menghitung jumlah gejala yang cocok (`cocok`) dari total gejala yang dimiliki kerusakan tersebut (`total`). Perhitungan dilakukan dengan `sum(k in jawaban_user for k in data["kode"])`. Hasil dari semua kerusakan dikumpulkan dalam list `skor_kerusakan`.

Penanganan Ketidakcocokan

Program menangani dua skenario:

1. Jika ada satu atau lebih kerusakan yang memiliki `cocok == total` (kecocokan sempurna 100%), maka semua kerusakan tersebut ditampilkan beserta solusinya.
2. Jika tidak ada yang sempurna, program mencari nilai `cocok` tertinggi (`max_cocok`). Semua kerusakan yang memiliki nilai `cocok` sama dengan `max_cocok` akan ditampilkan, lengkap dengan tingkat kecocokan dalam persen dan solusi masing-masing. Jika `max_cocok` bernilai 0 (tidak ada gejala yang cocok dengan aturan mana pun), program menampilkan pesan "Tidak ada gejala yang cocok dengan basis pengetahuan."

Dengan pendekatan ini, program tetap informatif meskipun pengguna tidak memberikan gejala yang persis sesuai dengan aturan manapun.

Output

Hasil diagnosa ditampilkan dalam kotak dialog `messagebox`. Output selalu mencakup nama kerusakan yang teridentifikasi (atau beberapa kerusakan jika ada lebih dari satu yang cocok) dan solusi singkat yang dapat dilakukan pengguna. Untuk kasus kecocokan parsial, juga ditampilkan persentase tingkat keyakinan.

Antarmuka Pengguna (GUI)
GUI dibuat dengan Tkinter dan terdiri dari:
- Label judul program.
- Label informasi yang menunjukkan nomor pertanyaan.
- Label pertanyaan dengan latar putih dan bingkai untuk menampilkan teks gejala.
- Dua tombol "Ya" dan "Tidak" yang muncul setelah tombol "Mulai Diagnosa" ditekan.
- Tombol "Mulai Diagnosa" untuk memulai sesi baru dan mereset state.

Logika tampilan: saat program pertama kali dijalankan, hanya tombol "Mulai Diagnosa" yang terlihat. Setelah tombol tersebut diklik, tombol "Ya" dan "Tidak" muncul, lalu pertanyaan pertama ditampilkan. Setiap jawaban akan memajukan indeks pertanyaan hingga semua gejala selesai ditanyakan. Setelah itu, fungsi `tampilkan_hasil()` dipanggil untuk melakukan inferensi dan menampilkan hasil. Selanjutnya program kembali ke keadaan awal dengan tombol "Mulai Diagnosa" siap untuk sesi baru.